

Fast Networks for High-Performance Distributed Trust

Yicheng Liu UCLA easonliu@cs.ucla.edu	Rafail Ostrovsky UCLA rafail@cs.ucla.edu	Scott Shenker UC Berkeley and ICSI shenker@icsi.berkeley.edu
Sam Kumar UCLA samkumar@cs.ucla.edu		

Abstract

Organizations increasingly need to collaborate by performing a computation on their combined dataset, while keeping their data hidden from each other. Certain kinds of collaboration, such as collaborative data analytics and AI, require a level of performance beyond what current cryptographic techniques for distributed trust can provide. This is because the organizations run software in *different trust domains*, which can require them to communicate over WANs or the public Internet. In this paper, we explore how to instead run such applications using fast datacenter-type LANs. We show that, by carefully redesigning distributed trust frameworks for LANs, we can achieve up to order-of-magnitude better performance than naïvely using a LAN. Then, we develop deployment models for *Distributed But Proximate Trust (DBPT)* that allow parties to use a LAN while remaining physically and logically distinct. These developments make secure collaborative data analytics and AI significantly more practical and set new research directions for developing systems and cryptographic theory for high-performance distributed trust.

1 Introduction

Distributed trust is a technique for building efficient systems with cryptographic security properties. With distributed trust, n different parties work together to run a system, and the system’s security guarantees hold if up to m of those parties (for some m, n such that $1 \leq m \leq n-1$) are compromised by an adversary. It enables applications like anonymous communication [20, 26, 68], metadata-hiding file sharing [22, 23], permissioned blockchains [6], private blocklists [64], and privacy-preserving statistics collection [1, 14, 25].

Certain promising distributed trust applications are not widely deployed because they require a level of performance that is beyond what current techniques can provide. We refer to them as *high-performance distributed trust* applications, or HPDTs, and they are the focus of this paper. One example of an HPDT is collaborative data analytics [7, 41, 54, 105], in which multiple organizations perform data analysis on their combined dataset, while keeping their data hidden from each other. Other examples are AI model training on a combined dataset [94, 109] and privacy-preserving AI inference [56, 70, 82, 83, 89, 92]. HPDTs have drawn industry interest, including from the financial [38] and healthcare [85] sectors, and have incipient adoption such as in Meta’s Private Lift [91] and Google’s Private Join and Compute [106]. Still, HPDT adoption remains limited and isolated, due to performance barriers seemingly inherent in distributed trust.

The performance cost of distributed trust stems not only from its cryptography, but also from its deployment model. In a distributed-trust system, each party must be deployed in a *different trust domain*, with no central point of attack or single administrative party spanning multiple trust domains [30]. In distributed trust deployments in industry, each party often hosts their component of the system in their own computing infrastructure (or *internal cloud*) [86, 103]. If a distributed trust application is run in a public cloud, it is often seen as a requirement that parties are in *different* clouds (e.g., one party in Amazon AWS and another party in Microsoft Azure) so that no single cloud provider becomes a central point of attack [24, 44, 59, 67, 74, 78], or at the very least in different regions of the same cloud [23, 25, 70, 82, 108]. Thus, the n parties may have to communicate using **wide-area networks (WANs) or the public Internet**.

This is costly [74] because the underlying cryptography often requires the n parties to exchange very large amounts of data or exchange data over many round trips, proportional to the size of the computation. The cost is not only in performance, but also monetary, as large data transfers incur high egress costs in the cloud [55, 87]. For HPDTs, these costs are compounded by increasing sizes of datasets and AI models.

Today, our community assumes that, for distributed trust, parties may have to communicate via WANs or the public Internet. Researchers routinely measure empirical performance with parties in distinct geographic regions or cloud regions (or with networks that emulate this) [23, 31, 37, 67, 70, 82, 89, 100, 108, 109, 117]. Communication and round complexity are defining performance metrics in cryptographic protocol design [21, 37, 40, 89, 100], and support for geographically distant parties is seen as a hallmark of scalability [108].

Certain HPDT systems use security models in which one trusted administrator [2, 13, 98] or cloud provider [40] controls all parties’ infrastructures, e.g., in one datacenter. Systems researchers/practitioners view this as weaker than true distributed trust [30, 59, 74], and we share this view. For example, the parties could just have the administrator run the entire application on their behalf (e.g., via a clean room [4, 29, 46, 99]), without cryptographic HPDT protocols. We suspect that such weaker security models have arisen because HPDTs perform far better over LANs than over WANs. Indeed, security researchers routinely measure HPDT protocols over LANs as a

point of comparison [19, 56, 77, 82, 83, 90, 92, 109], but without discussing implications for distributed trust. The missing piece is an architecture for deploying HPDTs in datacenter-type LANs while retaining true distributed trust.

This paper aims to supply that missing piece. We begin by showing that designing HPDT software specifically for fast LANs is a compelling research direction that can yield performance gains of up to an order of magnitude in widely used frameworks like EMP-Toolkit [107] and MP-SPDZ [60] (§3). These performance gains directly apply to systems that use weaker security discussed above (§4). To realize these performance gains *without* sacrificing distributed trust, we develop architectures for **Distributed But Proximate Trust (DBPT)** that enable widespread deployment of HPDTs using fast LANs (§5). We then outline research directions to further accelerate HPDTs in such architectures (§6).

Our message to the community is to: (1) treat fast LANs as a primary target for deploying HPDTs, (2) further develop DBPT architectures for deploying HPDTs in LANs, and (3) design and develop HPDT software to fully leverage fast networks. We hope that this agenda will remove performance obstacles that currently gate widespread adoption of HPDTs.

2 Background

2.1 Secure Multi-Party Computation

The full generality of distributed trust is captured by a family of cryptographic tools called Secure Multi-Party Computation (MPC) [11, 45, 114]. MPC enables n parties, each with secret data $x_1, \dots, x_n$, to compute $f(x_1, \dots, x_n)$, for any function f of their choice, while ensuring that no party learns anything about any other party’s secret data except for what is inherently revealed by the output of the function f .

In §3, we focus on *generic* MPC protocols capable of working with *any* function f . Generic MPC protocols work by expressing f as a boolean or arithmetic circuit C , where wires represent encrypted or secret-shared [97] values and gates represent operations on these values. These protocols are network-intensive because *communication among the parties is proportional to the size of C* . We next describe two types of generic MPC, focusing on how they use the network.

2.1.1 Garbled Circuits

In MPC protocols based on *garbled circuits* [12, 16, 79, 107, 113, 114], wires represent encrypted bits, and gates represent AND and XOR operations over those bits. Classically, garbled circuits work with $n = 2$ parties, called *garbler* and *evaluator*, but generalizations to more than 2 parties exist [9, 47, 49, 108]; in any case, all parties execute the circuit. Executing an XOR gate requires no communication and is concretely very fast [65]. Executing an AND gate (over two encrypted *bits*), however, requires the garbler to send 32 bytes of data (two ciphertexts of 128 bits each) to the evaluator [115]. Data transfer is in one direction (i.e., one network round) and the data can be streamed concurrently with circuit execution [52]. Thus, garbled-circuit-based MPC is *bandwidth-intensive* in network use.

2.1.2 Secret Sharing

In MPC protocols based on *secret sharing* [27, 28, 45, 60], wires represent secret-shared [97] values over an algebraic structure (e.g., integers modulo a prime), and gates represent addition and multiplication operations over those values. Executing an addition gate requires no communication. Executing a multiplication gate requires the parties to exchange data. The network round must complete before the output of the multiplication gate is obtained, so the total number of network rounds is the depth of the circuit in multiplication gates. Thus, secret-sharing-based MPC is *round-intensive* in how it uses the network.

2.2 Network Costs of MPC

HPDTs are often deployed with parties communicating over WANs or the public Internet. Such networks have high latency (e.g., milliseconds to 100s of milliseconds). This limits performance for secret-sharing-based MPC, for which each network round experiences a full RTT of latency. Additionally, TCP often achieves only limited bandwidth over such networks. This degrades performance for garbled-circuit-based MPC, which can become bandwidth-bound.

In principle, it is possible to sidestep bandwidth-related performance issues, since large amounts of bandwidth *exist* over the wide area. For example, one can use multiple parallel TCP connections to obtain higher bandwidth for MPC [67]. Alternatively, one could statically allocate bandwidth for MPC over a private WAN (e.g., prioritizing MPC traffic over other traffic sources). Unfortunately, such techniques have a significant monetary cost. Large data transfers incur high data egress costs in the cloud [55, 87], and purchasing large amounts of wide-area network bandwidth is expensive. Fundamentally, it is expensive to build and deploy high-bandwidth WANs [50], and the costs are passed on to users.

2.3 Alternatives to MPC

We briefly discuss two alternative approaches to enabling HPDTs, to explain why MPC is preferable in certain cases.

2.3.1 Fully Homomorphic Encryption (FHE)

FHE allows general computation on encrypted data. Given the encryptions of n items $x_1, \dots, x_n$, FHE allows one to compute the encryption of $f(x_1, \dots, x_n)$ for any function f , without knowing the secret key. Unlike MPC, computing f using FHE does not require communication among parties, except for providing the input at the start and decrypting the output at the end.

Unfortunately, FHE usually cannot be used as a replacement for MPC. The reason is that FHE is *three to four (or more) orders of magnitude more computationally expensive* than MPC. Whereas the CPU time to execute MPC gates is measured in nanoseconds or microseconds, FHE operation times are measured in milliseconds [66, 104]. For example, the “bootstrap” FHE operation, necessary for executing circuits with high multiplicative depth, takes several milliseconds to complete at best [72]. That said, in settings where network communication is very limited or expensive, FHE may still be preferable to MPC, despite its computational overhead, because it uses the network much more sparingly.

2.3.2 Trusted Execution Environments (TEEs)

TEEs, like Intel SGX/TDX and AMD SEV-SNP, are a hardware feature that allows software to run in an environment isolated even from kernel and hypervisor software [8]. They work by cryptographically securing the TEE’s data while it is in memory, decrypting data as they enter the CPU caches. Their security assumes a trusted hardware design that must not expose TEEs’ secret keys to any software, and a silicon root of trust for attesting that the intended code is running in the TEE.

TEEs have been proposed as an alternative to MPC for HPDTs [116]. The idea is to collect all parties’ data within a TEE, allowing computation on all parties’ data to take place within the TEE. The TEE is hosted at a single party, but the TEE’s security guarantees prevent that party from directly observing or influencing the data inside the TEE.

TEEs, however, have a major downside compared to MPC: They have been found, time and time again during their short history, to be susceptible to security attacks, particularly via side channels [15, 17, 69, 71, 84, 101, 102]. Such attacks potentially allow the party hosting the TEE to bypass its protections and access data inside the TEE, compromising the other parties’ data. Given the complexity of modern CPUs, it may not be possible to make them totally side-channel free.

3 MPC System Design for Fast LANs

Researchers measure performance of their distributed-trust systems not only in WANs, but also in LANs, usually as a point of comparison [32, 40, 61, 67, 90, 108, 109]. Depending on the nature of the application and WAN setup, the performance gain from using a LAN instead of a WAN has been shown to be significant—for example, up to nearly $10\times$ [109], with increasing speedups for $n > 2$ parties [108]. RDMA has been shown to provide $1.2\text{--}4.2\times$ additional performance [90].

However, there is a difference between just running HPDT software in a LAN, and properly designing HPDT software to fully benefit from LANs. In this section, we initiate the study of how to build HPDT systems that can fully leverage fast LANs. We present initial work that shows an *additional* $10\times$ improvement via lightly redesigned MPC frameworks.

As an analogy, consider how the emergence of kernel-bypass networking initiated a rich line of research on how to best use it in regular (i.e., non-distributed-trust) datacenter systems and applications [3, 18, 57, 58, 63, 96, 110, 112]. Fast networks for HPDTs (e.g., enabled by DBPT architectures in §5) similarly raise the bar for HPDT systems.

3.1 Experimental Methodology

We use two single-socket servers. Each server has an Intel Xeon Gold 5520+ Emerald Rapids CPU, a 10 Gbps Intel X710-T2L NIC (X710), and a 200 Gbps Nvidia MCX755106ASA-HEAT ConnectX-7 NIC (CX-7). The X710 NICs are connected via an RJ45 cable, and the CX-7 NICs are connected via a QSFP56 Direct Attach Copper cable. In both cases, the NICs are directly connected, with no intervening switches.

As a baseline for HPDTs running over a LAN representative of prior work, we use the Linux network stack with the X710 NICs. For the X710 NICs, `ping` reports an RTT of $\approx 800\ \mu\text{s}$. While it is possible to obtain more than 10 Gbps locally or over the public Internet, the 10 Gbps provided by X710 is sufficient for our purposes; a modern garbled circuit protocol [107, 115] (bandwidth-intensive type of MPC) with one CPU core per party does not saturate a 10 Gbps link [67].

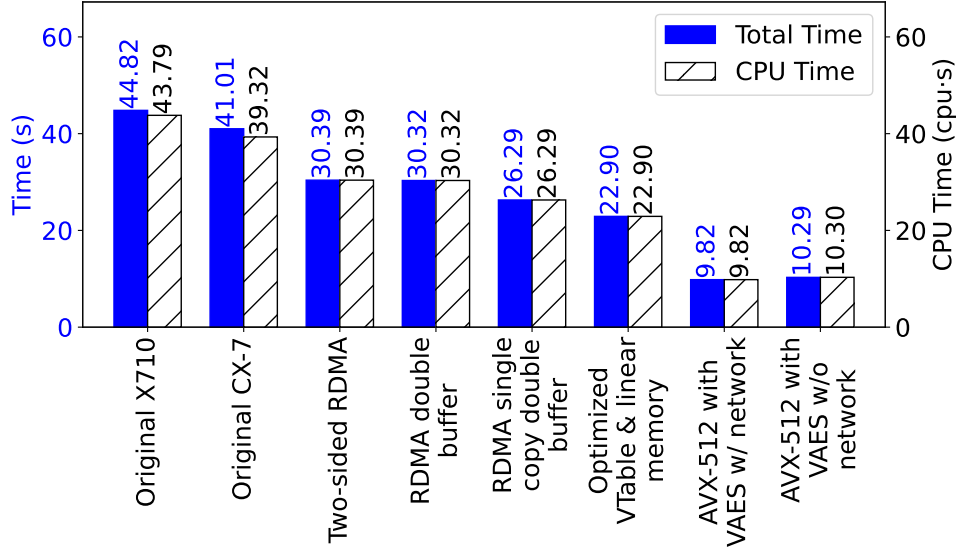


Figure 1: Total time and CPU time for EMP-Toolkit.

For a setup optimized for LAN environments, we use RDMA with the CX-7 NICs. For the CX-7 NICs, `ib_send_lat` reports a typical one-way (e.g., half-RTT) latency of $\approx 1 \mu\text{s}$ for two-sided RDMA, and `ib_read_lat` and `ib_write_lat` report typical one-way latencies (e.g., half-RTTs) of $\approx 2 \mu\text{s}$ and $\approx 1 \mu\text{s}$, respectively, for one-sided RDMA.

For each type of MPC, for the function f to run, we use multiplication of a 1024×1024 matrix with a 1024-element vector, where one party provides the matrix and the other party provides the vector. We measure baseline performance with the X710 NICs, representative of LAN experiments in prior work. Then, we switch to the CX-7 NICs and iteratively redesign the software to better benefit from the LAN setting.

3.2 Garbled-Circuit-Based MPC

We study garbled-circuit-based MPC using the widely used EMP-Toolkit framework [107], in a semi-honest threat model. We run our workload with the matrix and vector containing 32-bit integers. Fig. 1 shows the wall-clock time and CPU time for each setup. Each setup builds on the one to its left.

As shown in Fig. 1, simply upgrading from the commonly used 10 Gbps (X710) NIC to a 200 Gbps (CX-7) NIC does not significantly improve performance. This is because the program is not bottlenecked by available network bandwidth. However, using RDMA via `ibverbs`, instead of the Linux network stack, significantly improves performance. This is because RDMA eliminates the CPU overhead of the Linux network stack, allowing more CPU time to be spent on cryptographic work of the garbled circuit protocol. Optimizations to use RDMA more efficiently, such as double buffering and single-copy, further enhance performance.

Once we use RDMA to eliminate Linux network stack overheads, other software overheads that were previously overshadowed by the cost of using the network are now significant. For example, for software extensibility purposes, EMP-Toolkit implements AND and XOR gates as virtual function

calls. Eliminating these virtual function calls in favor of template-based static polymorphism, together with using an inlined memory layout, results in a noticeable speedup.

Similarly, AND gates require frequent hash operations [48], primarily based on AES. To improve performance, we changed EMP-Toolkit from using AES-NI instructions [10] to using VAES instructions. Fully utilizing VAES hardware required a new vectored/batched abstraction for programming f . In total, the techniques bring $\approx 4.5\times$ speedup over the original.

Finally, we completely remove use of the network (which affects correctness) and observe that, even then, performance does not improve. This suggests that further optimizing how we use RDMA is unlikely to yield more performance gains.

3.3 Secret-Sharing-Based MPC

We now study secret-sharing MPC using the “mod 2^k ” algorithm in the MP-SPDZ framework [60], in a semi-honest threat model. We run our workload with the matrix and vector containing 64-bit integers ($k = 64$); in MP-SPDZ, it incurs many network rounds. Fig. 2 shows the wall-clock time and CPU time, including both online and preprocessing phases.

“Two-sided RDMA” in Fig. 2 includes the RDMA-related optimizations in §3.2. However, its performance is worse than using the Linux network stack. This is because, whereas garbled circuits are *bandwidth-intensive*, secret-sharing MPC is *round-intensive* (§2.1). As a result, our design focus must shift from maximizing throughput to minimizing latency. Accordingly, we replaced the RDMA buffering system from §3.2 with a queue-based system: Each new piece of data triggers a new RDMA send request, which is matched with a receive request at the recipient. We also switched from using two-sided RDMA to using one-sided RDMA, reducing latency and CPU overhead associated with synchronization.

MP-SPDZ originally used a threading model that assumed that network latency is much higher than the cost of thread switching and synchronization. But, in our optimized LAN setting, this assumption no longer holds. Thus, we redesigned the threading model, switching from a multi-threaded execution model to a single-threaded execution model.

Overall, these techniques achieve $\approx 10\times$ speedup—an order of magnitude—over the original version, and $\approx 40\times$ speedup compared to the first RDMA version.

3.4 Discussion and Future Opportunities

In our efforts above, just using RDMA instead of the Linux network stack provides a marginal improvement, at best. We believe the reason is that existing MPC frameworks were designed assuming slower networks. Once we applied kernel bypass to fix network bottlenecks, new bottlenecks emerged. Thus, RDMA serves as a hardware foundation for optimization, enabling subsequent software design changes to yield up to an order-of-magnitude performance improvement.

An opportunity to further accelerate MPC over fast LANs is parallel execution using multiple CPU cores [16]. EMP-Toolkit and MP-SPDZ, despite being widely used, do not parallelize circuit execution across multiple cores. This may be because they were designed assuming the network is the bottleneck, so that using multiple cores would not help. This is no longer the case over a fast LAN. In our experiments above, with a single core per party, our optimized EMP-Toolkit uses ≈ 30 Gbps of unidirectional bandwidth and our optimized MP-SPDZ uses ≈ 5 Gbps of bidirectional bandwidth, out of 200 Gbps provided by the NIC. This suggests that parallel execution is a promising performance opportunity.

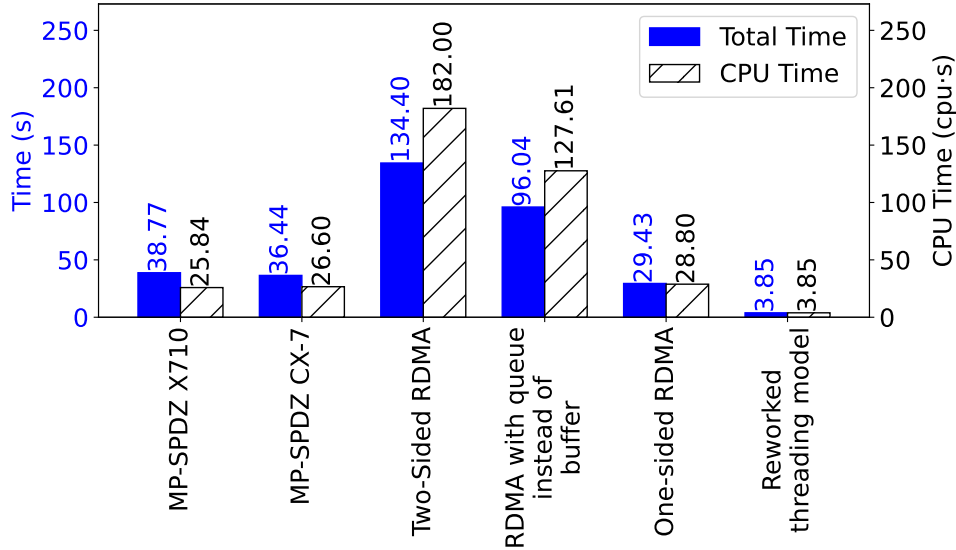


Figure 2: Total time and CPU time for MP-SPDZ.

Another opportunity is to design the machine shape. For example, the ≈ 30 Gbps required by our optimized single-core garbled circuit execution suggests that, with a 200 Gbps NIC, parallelism will not scale beyond ≈ 6 – 7 cores. We can solve this by installing *multiple* ConnectX-7 NICs within each server. For example, a server with 64 PCIe 5.0 lanes available for networking can theoretically support ≈ 2 Tbit/s. This could be scaled further by using multiple CPU sockets.

Overall, we optimistically estimate up to *three orders of magnitude* performance gains in moving from a WAN to a fast LAN—one identified by prior work (top of §3), one from our work in §3.2 and §3.3, and one from parallelization across multiple cores and adjusting machine shape (§3.4). This motivates DBPT architectures that would enable HPDTs to benefit from fast LANs while maintaining distributed trust.

4 Trusted Administrative Entities

In certain industry deployments, it is considered sufficient for the n parties to execute on physically different servers, even if a single administrator controls all parties [98] or their infrastructure [2, 13]. Certain academic works target this model too, assuming the parties use machines controlled by separate cloud accounts in the same datacenter [40].

Our position, in this paper, is that *such deployment models do not truly provide distributed trust*. This view is echoed by systems researchers and industry practitioners working on distributed trust [30, 59, 74]. For example, if both parties are in the same cloud datacenter, a malicious cloud employee could see both parties’ memories and subvert MPC entirely. Furthermore, if all parties trust a single cloud provider, they could use a clean room [4, 29, 46, 99] instead of MPC.

Still, we note that such deployments, with a mutually trusted administrator or cloud, can benefit from the research we proposed in §3. They would just need to use optimized MPC software, and server and network hardware capable of kernel bypass (e.g., RDMA). Cloud providers would have

to allow accounts controlled by *different* tenants to allocate kernel-bypass VMs that are placed nearby (e.g., on the same rack). To our knowledge, cloud providers today do not allow this, but we see no technical obstacles to them doing so.

5 Distributed but Proximate Trust

How can HPDT deployments benefit from careful system design for LANs, as we proposed in §3, without weakening distributed trust as in the models discussed in §4? In this section, we answer these questions by developing deployment models that enable parties to remain in separate trust domains while being physically near each other. We call such architectures *Distributed But Proximate Trust (DBPT)*. Security researchers routinely measure HPDT protocols over LANs [19, 56, 77, 82, 83, 90, 92, 109]; DBPT fills a valuable missing piece for such work, providing a basis for LAN-based deployments and grounding for LAN-based experiments.

We present our DBPT models as a sequence of thought experiments. First, we aim to understand if there are any fundamental roadblocks: *in principle*, with sufficient resources, can n parties achieve DBPT? We iterate on the resulting setup, moving toward designs that are more broadly accessible.

5.1 DBPT for Well-Resourced Parties

Let us assume that the n parties who wish to run an HPDT have the resources and expertise for significant investments in computing infrastructure. This would be representative of hyperscalars, like Microsoft and Amazon, wanting to run a high-value HPDT. Are there any *fundamental* obstacles that would prevent such parties from obtaining DBPT?

Our solution is for such parties to coordinate to *build datacenters that are physically nearby*—for example, in different buildings in the same city block. These need not be large facilities like traditional cloud regions, but can be small facilities similar to points of presence or edge datacenters. This is consistent with the expansion of cloud infrastructure to the edge via smaller datacenter deployments, such as Azure Edge/Extended Zones [62, 81] and AWS Local Zones [5].

The datacenters’ proximity enables connectivity via dedicated, high-bandwidth fiber optic cables, providing a fast network for HPDTs. Whereas a global-scale WAN is extremely expensive to build and run, datacenters in the same city block can be connected cheaply via commodity hardware. For example, a 200 Gbps fiber optic cable 150 m in length costs only ≈ 500 USD [42], making it feasible to connect each server in one datacenter one-to-one with a corresponding server in another party’s datacenter via *multiple* such cables, at only a fraction of the cost of the servers themselves. The datacenters’ physical proximity would enable network latencies comparable to intra-rack networks. For example, propagation delay over a 200 m fiber optic cable is only ≈ 1 μ s, comparable to processing delays within NICs [95, 111].

Each party can have disjoint sets of employees manage their respective datacenters and their buildings’ physical security. Thus, from an administration standpoint, there is no entity in control of both parties’ infrastructures. While there is still some “ambient trust,” such as trust in hardware vendors (e.g., Intel) and supply chains, this also applies to the status quo in which datacenters are not physically nearby.

Outside of computer administration/manufacturing, there is a minor difference in trust model: Nearby datacenters may share the same political authority, if located in the same country or municipality. If the parties are in different countries and do not trust each other’s government,

then physical co-location may weaken the trust model, unless the datacenters are co-located in a way that straddles a shared national border. That said, many applications of HPDTs involve parties in the same country, for which this would not be an issue.

5.2 DBPT via HPDT-Friendly Clouds

Not all organizations wishing to run HPDTs can directly use the deployment model in §5.1. For example, banks and hospitals stand to benefit from HPDTs, but may not have the resources or expertise to build/manage datacenters and run network cables. How can such organizations obtain DBPT?

Our observation is that, if c hyperscalars implement the deployment model in §5.1, then they can make it available to other, less-well-resourced parties while preserving trust boundaries. In effect, they can become “cloud providers” for a HPDT-friendly cloud that other parties can use.

The idea is that each of the n parties can use secret sharing to split its data into c shares, and then give each provider one of the shares. While distributing these shares requires network bandwidth, it is proportional only to the size of the data, not to the size of the computation, and therefore is likely to be much cheaper than just running MPC over the public Internet. This approach is directly inspired by prior work that separates compute providers, which act as parties in a distributed trust system, from data owners [23, 73].

In cryptography, this is sometimes called the “client-server model.” As long as the requisite number of compute providers are honest, the distributed trust guarantees of MPC hold, possibly with abort. This assumption is reasonable because the providers are well-resourced and economically motivated to not collude. Any evidence of misbehavior would cause significant economic damage due to reputational impact, likely outweighing the value of stealing the n parties’ data.

5.3 DBPT for Everyone

The approach in §5.2 has a major drawback: It requires at least two hyperscalars to invest in co-located datacenters and make them accessible to others via a cloud service. We are not aware of any such deployment today. How can organizations without resources or expertise to build datacenters obtain DBPT, in a way that is practical and deployable today?

Our insight is to use rentable datacenter and edge infrastructure distinct from the cloud. Companies like Digital Realty [33], Equinix [39], Hurricane Electric [53], Rackspace [88], etc. provide *colocation services* (“colos”). With colos, customers provide their own physical servers; the colo provides power and networking infrastructure to host their customers’ servers. Whereas cloud providers control hypervisor software running on each server, colo providers are only responsible for power and networking, and leave customers in full software control of their servers. This provides a path to use colos for HPDTs without the colo provider gaining control over all parties’ infrastructure.

Today, colos offer an alternative to on-premises solutions, enabling the control associated with bringing one’s own servers while amortizing the operational costs (e.g., power, cooling, networking, security) of running a datacenter [39]. Although it is not the typical use case of a colo, colos can be used to provide fast networks for clients whose servers are in the same building. For example, Digital Realty provides “Cross Connects” that allow for dedicated network connections between different customers’ physical servers [34].

A major drawback of a colo-based solution, however, is that the colo provider has physical access to both parties’ computing infrastructures. An adversarial colo provider could conduct side channel attacks to learn the parties’ secret data. A malicious colo provider could even install unauthorized hardware inside customers’ servers, e.g., to snoop on the memory bus [69] to learn the parties’ secrets.

We observe that this issue could be overcome via a new research direction on developing *tamper-proof cages* for running physical servers in untrusted environments. The idea would be to destroy any sensitive data on a server if its cage is compromised while it is running. For example, one could install sensors in a server that detect if the cage is compromised, and if so, send a high priority interrupt to the CPU, which would respond by immediately clearing all registers and memory. Alternatively, one could design a tamper-proof cage that simply cuts power to the device. If an even higher degree of assurance is required, the tamper-proof cage could physically damage the server (i.e., immolation [80]). A possible alternative to tamper-proof cages is for each organization to have an employee monitor their server in the colo environment (or hire a trusted guard company to do the monitoring).

In this paper, we leave the design and implementation of tamper-proof cages to future work, and we do not provide a recommendation as to which approach is “best.” We simply note that the “physical access” problem with using colos for DBPT can be plausibly solved by developing such devices.

6 Future Research and Conclusion

We urge the community to further develop DBPT architectures for deploying HPDTs in LANs, to realize the performance gains in §3. In particular, the DBPT model in §5.3 has the potential to work broadly, as it does not require the buy-in of hyperscalars. Research on tamper-proof cages can help enable this model, and thereby DBPT.

DBPT enables additional performance gains, beyond those in §3, because it shifts the balance between CPU and networking. Because DBPT makes high-bandwidth and low-latency communication cheaply available to HPDTs, one could potentially improve HPDT performance by reducing CPU usage at the expense of higher network usage (to an extent). For example, HPDT systems can adjust how they combine bandwidth-intensive garbled-circuit-based MPC and round-intensive secret-sharing-based MPC to strike a new balance. On the theory side, there is an opportunity to develop new distributed trust protocols, including for distributed ORAM [37, 40, 100], LPZK [35, 36], PSI [43, 51], secret-sharing-based MPC [11], etc., to find different trade-offs between CPU and networking. For example, a major focus in garbled-circuit-based MPC design over the past two decades has been to reduce the amount of data transferred between the parties [65, 75, 76, 93, 113, 115]; DBPT suggests making trade-offs in the opposite direction.

References

- [1] Surya Addanki, Kevin Garbe, Eli Jaffe, Rafail Ostrovsky, and Antigoni Polychroniadou. Prio+: Privacy preserving aggregate statistics via boolean shares. In *Security and Cryptography for Networks*, pages 516–539. Springer International Publishing, 2022.
- [2] Alchemy. Build anything onchain. <https://www.alchemy.com/>. Accessed: June 24, 2025.

- [3] Emmanuel Amaro, Zhihong Luo, Amy Ousterhout, Arvind Krishnamurthy, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. Remote memory calls. In *HotNets*. ACM, 2020.
- [4] Amazon. Data collaboration service - AWS clean rooms - AWS. <https://aws.amazon.com/clean-rooms/>. Accessed: September 6, 2024.
- [5] Amazon Web Services. AWS Local Zones: User guide. <https://docs.aws.amazon.com/pdfs/local-zones/latest/ug/local-zones.pdf#what-is-aws-local-zones>, 2025. Accessed: June 28, 2025.
- [6] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *EuroSys*. ACM, 2018.
- [7] Johes Bater, Gregory Elliott, Craig Eggen, Satyender Goel, Abel Kho, and Jennie Rogers. SMCQL: Secure querying for federated databases. *VLDB*, 10(6), 2017.
- [8] Andrew Baumann, Marcus Peinado, and Galen Hunt. Shielding applications from an untrusted cloud with Haven. In *OSDI*. USENIX, 2014.
- [9] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols. In *STOC*, page 503–513. ACM, 1990.
- [10] Mihir Bellare, Viet Tung Hoang, Sriram Keelveedhi, and Phillip Rogaway. Efficient garbling from a fixed-key blockcipher. In *S&P*, pages 478–492. IEEE, 2013.
- [11] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *STOC*, page 1–10. ACM, 1988.
- [12] Osman Biçer. Efficiency optimizations on Yao’s garbled circuits and their practical applications. Master’s thesis, Istanbul Şehir University, 2017. Chapters 3 and 4.
- [13] Blockdaemon. Blockdaemin — institutional gateway to Web3. <https://www.blockdaemon.com/>. Accessed: June 24, 2025.
- [14] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Lightweight techniques for private heavy hitters. In *S&P*. IEEE, 2021.
- [15] Pietro Borrello, Andreas Kogler, Martin Schwarzl, Moritz Lipp, Daniel Gruss, and Michael Schwarz. $\mathcal{A}$ EPIC leak: Architecturally leaking uninitialized data from the microarchitecture. In *USENIX Security*, pages 3917–3934, Boston, MA, August 2022. USENIX Association.
- [16] Niklas Buescher and Stefan Katzenbeisser. Faster secure computation through automatic parallelization. In *USENIX Security*. USENIX, 2015.
- [17] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the Intel SGX kingdom with transient Out-of-Order execution. In *USENIX Security*, page 991–1008, Baltimore, MD, August 2018. USENIX Association.

- [18] Matthew Burke, Sowmya Dharanipragada, Shannon Joyner, Adriana Szerkeres, Jacob Nelson, Irene Zhang, and Dan R. K. Ports. PRISM: Rethinking the RDMA interface for distributed systems. In *SOSP*. ACM, 2021.
- [19] Nishanth Chandran, Divya Gupta, Aseem Rastogi, Rahul Sharma, and Shardul Tripathi. EzPC: Programmable and efficient secure two-party computation for machine learning. In *EuroS&P*, pages 496–511, 2019.
- [20] David L. Chaum. Untraceable electronic mail, return addresses, and digital pseudonyms. *Commun. ACM*, 24(2):84–90, 1981.
- [21] Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In *CCS*, page 1243–1255. ACM, 2017.
- [22] Weikeng Chen, Thang Hoang, Jorge Guajardo, and Atilla A. Yavuz. Titanium: A metadata-hiding file-sharing system with malicious security. In *NDSS*. Internet Society, 2022.
- [23] Weikeng Chen and Raluca Ada Popa. Metal: A metadata-hiding file-sharing system. In *NDSS*. Internet Society, 2020.
- [24] Graeme Connell, Vivian Fang, Rolfe Schmidt, Emma Dauterman, and Raluca Ada Popa. Secret key recovery in a Global-Scale End-to-End encryption system. In *OSDI*, pages 703–719, Santa Clara, CA, July 2024. USENIX Association.
- [25] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *NSDI*. USENIX Association, 2017.
- [26] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *S&P*, pages 321–338. IEEE, 2015.
- [27] Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority – or: Breaking the SPDZ limits. In *ESORICS*. Springer, 2013.
- [28] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*. Springer, 2012.
- [29] Databricks. Databricks clean rooms — databricks. <https://www.databricks.com/product/clean-room>. Accessed: September 8, 2024.
- [30] Emma Dauterman, Vivian Fang, Natacha Crooks, and Raluca Ada Popa. Reflections on trusting distributed trust. In *HotNets*. ACM, 2022.
- [31] Emma Dauterman, Mayank Rathee, Raluca Ada Popa, and Ion Stoica. Waldo: A private time-series database from function secret sharing. In *S&P*, pages 2450–2468. IEEE, 2022.
- [32] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY - a framework for efficient mixed-protocol secure two-party computation. In *NDSS*. Internet Society, 2015.
- [33] DigitalRealty. Colocation provider — PlatformDIGITAL — Digital Realty. <https://www.digitalrealty.com/platform-digital/colocation>, n.d. Accessed: June 29, 2025.

- [34] DigitalRealty. Cross connect — Digital Realty. <https://www.digitalrealty.com/platform-digital/connectivity/interconnection/cross-connect>, n.d. Accessed: June 29, 2025.
- [35] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Improving line-point zero knowledge: Two multiplications for the price of one. In *CCS*, CCS '22, page 829–841, New York, NY, USA, 2022. ACM.
- [36] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-Point Zero Knowledge and Its Applications. In *ITC*, pages 5:1–5:24. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021.
- [37] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In *CCS*. ACM, 2017.
- [38] Erez Eizenman. Scotiabank’s chief risk officer on the state of anti-money laundering. <https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/scotiabanks-chief-risk-officer-on-the-state-of-anti-money-laundering>, 2019. Accessed: June 23, 2025.
- [39] Equinix. Colocation at Equinix. <https://www.equinix.com/products/data-center-services/colocation>, n.d. Accessed: June 29, 2025.
- [40] Brett Falk, Rafail Ostrovsky, Matan Shtepel, and Jacob Zhang. GigaDORAM: Breaking the billion address barrier. In *USENIX Security*. USENIX Association, 2023.
- [41] Brett Hemenway Falk, Steve Lu, and Rafail Ostrovsky. DURASIFT: A robust, decentralized, encrypted database supporting private searches with complex policy controls. In *WPES*, page 26–36. ACM, 2019.
- [42] FiberMall. QSFP56-200G-AOC-150M 150m (492ft) 200G QSFP56 to QSFP56 active optical cable. <https://www.fibermall.com/sale-459089-qsfp56-200g-aoc-150m-active-optical-cable.htm>, n.d. Accessed: June 28, 2025.
- [43] Michael J. Freedman, Kobbi Nissim, and Benny Pinkas. Efficient private matching and set intersection. In *EUROCRYPT*. Springer, 2004.
- [44] Tim Geoghegan. Exposure notifications private analytics: Lessons learned from running secure mpc at scale. <https://divviup.org/blog/lessons-from-running-mpc-at-scale/>, May 2022. Accessed: June 24, 2025.
- [45] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game, or a completeness theorem for protocols with honest majority. In *STOC*. ACM, 1987.
- [46] Google. Secure and privacy-centric sharing with data clean rooms in BigQuery. <https://cloud.google.com/blog/products/data-analytics/introducing-bigquery-data-clean-rooms>. Accessed: September 8, 2024.
- [47] Vipul Goyal, Junru Li, Rafail Ostrovsky, and Yifan Song. Towards building scalable constant-round MPC from minimal assumptions via round collapsing. In *CRYPTO*. IACR, 2025.

- [48] Chun Guo, Jonathan Katz, Xiao Wang, Chenkai Weng, and Yu Yu. Better concrete security for half-gates garbling (in the multi-instance setting). In *CRYPTO*, pages 793–822. Springer International Publishing, 2020.
- [49] David Heath, Vladimir Kolesnikov, Varun Narayanan, Rafail Ostrovsky, and Akash Shah. Multiparty garbling from OT with linear scaling and RAM support. In *CRYPTO*. IACR, 2025.
- [50] Chi-Yao Hong, Subhasree Mandal, Mohammad Al-Fares, Min Zhu, Richard Alimi, Kondapa Naidu B., Chandan Bhagat, Sourabh Jain, Jay Kaimal, Shiyu Liang, Kirill Mendelev, Steve Padgett, Faro Rabe, Saikat Ray, Malveeka Tewari, Matt Tierney, Monika Zahn, Jonathan Zolla, Joon Ong, and Amin Vahdat. B4 and after: managing hierarchy, partitioning, and asymmetry for availability and scale in Google’s software-defined wan. In *SIGCOMM*, page 74–87. Association for Computing Machinery, 2018.
- [51] Yan Huang, David Evans, and Jonathan Katz. Private Set Intersection: Are garbled circuits better than custom protocols? In *NDSS*. Internet Society, 2012.
- [52] Yan Huang, David Evans, Jonathan Katz, and Lior Malka. Faster secure two-party computation using garbled circuits. In *USENIX Security*. USENIX, 2011.
- [53] Hurricane Electric. Colocation - Hurricane Electric Internet services. <https://he.net/colocation.html>, n.d. Accessed: June 29, 2025.
- [54] Mihaela Ion, Ben Kreuter, Ahmet Erhan Nergiz, Sarvar Patel, Mariana Raykova, Shobhit Saxena, Karn Seth, David Shanahan, and Moti Yung. On deploying secure computing: Private intersection-sum-with-cardinality protocols. In *EuroS&P*. IEEE, 2020.
- [55] Paras Jain, Sam Kumar, Sarah Wooders, Shishir G. Patil, Joseph E. Gonzalez, and Ion Stoica. Skyplane: Optimizing transfer cost and throughput using Cloud-Aware overlays. In *NSDI*. USENIX Association, 2023.
- [56] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *USENIX Security*. USENIX, 2018.
- [57] Anuj Kalia, Michael Kaminsky, and David Andersen. Datacenter RPCs can be general and fast. In *NSDI*, pages 1–16, Boston, MA, February 2019. USENIX Association.
- [58] Anuj Kalia, Michael Kaminsky, and David G. Andersen. FaSST: Fast, scalable and simple distributed transactions with Two-Sided (RDMA) datagram RPCs. In *OSDI*, pages 185–201, Savannah, GA, November 2016. USENIX Association.
- [59] Darya Kaviani, Sijun Tan, Pravein Govindan Kannan, and Raluca Ada Popa. Flock: A framework for deploying On-Demand distributed trust. In *OSDI*, pages 721–743. USENIX Association, July 2024.
- [60] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *CCS*, CCS ’20, page 1575–1590. ACM, 2020.

- [61] Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT*. Springer International Publishing, 2018.
- [62] Yousef Khalidi. Microsoft partners with the industry to unlock new 5G scenarios with Azure Edge Zones. <https://azure.microsoft.com/en-us/blog/microsoft-partners-with-the-industry-to-unlock-new-5g-scenarios-with-azure-edge-zones/>, 3 2020. Accessed: June 28, 2025.
- [63] Jongyul Kim, Insu Jang, Waleed Reda, Jaeseong Im, Marco Canini, Dejan Kostić, Youngjin Kwon, Simon Peter, and Emmett Witchel. LineFS: Efficient SmartNIC offload of a distributed file system with pipeline parallelism. In *SOSP*. ACM, 2021.
- [64] Dmitry Kogan and Henry Corrigan-Gibbs. Private blocklist lookups with checklist. In *USENIX Security*. USENIX Association, 2021.
- [65] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In *ICALP*. Springer, 2008.
- [66] Sam Kumar. *Rethinking System Design for Expressive Cryptography*. PhD thesis, University of California, Berkeley, 2023.
- [67] Sam Kumar, David E. Culler, and Raluca Ada Popa. MAGE: Nearly zero-cost virtual memory for secure computation. In *OSDI*. USENIX, 2021.
- [68] Albert Kwon, David Lu, and Srinivas Devadas. XRD: Scalable messaging system with cryptographic privacy. In *NSDI*, pages 759–776, Santa Clara, CA, 2020. USENIX Association.
- [69] Dayeol Lee, Dongha Jung, Ian T. Fang, Chia che Tsai, and Raluca Ada Popa. An Off-Chip attack on hardware enclaves via the memory bus. In *USENIX Security*, pages 487–504. USENIX Association, August 2020.
- [70] Ryan Lehmkuhl, Pratyush Mishra, Akshayaram Srinivasan, and Raluca Ada Popa. MUSE: Secure inference resilient to malicious clients. In *USENIX Security*. USENIX Association, 2021.
- [71] Mengyuan Li, Yinqian Zhang, Huibo Wang, Kang Li, and Yueqiang Cheng. CIPHERLEAKS: Breaking constant-time cryptography on AMD SEV via the ciphertext side channel. In *USENIX Security*, pages 717–732. USENIX Association, August 2021.
- [72] Zhihao Li, Xianhui Lu, Zhiwei Wang, Ruida Wang, Ying Liu, Yinhang Zheng, Lutan Zhao, Kunpeng Wang, and Rui Hou. Faster ntru-based bootstrapping in less than 4 ms. *Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):418–451, July 2024.
- [73] John Liagouris, Vasiliki Kalavri, Muhammad Faisal, and Mayank Varia. SECRECY: Secure collaborative analytics in untrusted clouds. In *NSDI*. USENIX Association, 2023.
- [74] Yehuda Lindell, David Cook, Tim Geoghegan, Sarah Gran, Rolfe Schmidt, Ehren Kret, Darya Kaviani, and Raluca Ada Popa. The deployment dilemma: Merits & challenges of deploying MPC. <https://mpc.cs.berkeley.edu/blog/deployment-dilemma.html>, 9 2023. Accessed: July 5, 2025.

- [75] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. Authenticated BitGC for actively secure rate-one 2PC. In *CRYPTO*. IACR, 2025.
- [76] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. BitGC: Garbled circuits with 1 bit per gate. In *EUROCRYPT*. Springer, 2025.
- [77] Jian Liu, Mika Juuti, Yao Lu, and N. Asokan. Oblivious neural network predictions via MiniONN transformations. In *CCS*, page 619–631. ACM, 2017.
- [78] Joshua Lund. Technology preview for secure value recovery. <https://signal.org/blog/secure-value-recovery/>, 12 2019. Accessed: July 5, 2025.
- [79] Dahlia Malkhi, Noam Nisan, Benny Pinkas, and Yaron Sella. Fairplay—a secure two-party computation system. In *USENIX Security*. USENIX, 2004.
- [80] James Mickens, Sarah Radway, and Ravi Netravali. Guillotine: Hypervisors for isolating malicious AIs. In *HotOS XX*. ACM, 2025.
- [81] Microsoft. What are Azure Extended Zones. <https://learn.microsoft.com/en-us/azure/extended-zones/overview>, 5 2025. Accessed: June 28, 2025.
- [82] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. DELPHI: A cryptographic inference service for neural networks. In *USENIX Security*. USENIX Association, 2020.
- [83] Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *SP*. IEEE, 2017.
- [84] Kit Murdock, David Oswald, Flavio D. Garcia, Jo Van Bulck, Frank Piessens, and Daniel Gruss. Plundervolt: How a little bit of undervolting can create a lot of trouble. IEEE, 2020.
- [85] Owkin. Bridging federated learning theory and practice with real-world healthcare data. <https://www.owkin.com/blogs-case-studies/bridging-the-gap-between-federated-learning-theory-and-practice-with-real-world-healthcare-datasets>, 2022. Accessed: June 23, 2025.
- [86] Antigoni Polychroniadou, Gilad Asharov, Benjamin Diamond, Tucker Balch, Hans Buehler, Richard Hua, Suwen Gu, Greg Gimler, and Manuela Veloso. Prime Match: A privacy-preserving inventory matching system, 2023.
- [87] Matthew Prince and Nitin Rao. AWS’s egregious egress. <https://blog.cloudflare.com/aws-egregious-egress/>, 2021. Accessed: June 23, 2025.
- [88] Rackspace. Data center colocation services: Maintain control of your hardware, while cutting costs. <https://www.rackspace.com/cloud/colocation>, n.d. Accessed: June 29, 2025.
- [89] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CryptFlow2: Practical 2-party secure inference. In *CCS*, page 325–342. ACM, 2020.

- [90] Zhenghang Ren, Mingxuan Fan, Zilong Wang, Junxue Zhang, Chaoliang Zeng, Zhicong Huang, Cheng Hong, and Kai Chen. Accelerating secure collaborative machine learning with Protocol-Aware RDMA. In *USENIX Security*. USENIX Association, 2024.
- [91] James Reyes. Building the next generation of digital advertising with MPC. Real World Crypto, 2022. <https://youtu.be/6Gb0x08csVU?t=2533>.
- [92] M. Sadegh Riazi, Mohammad Samragh, Hao Chen, Kim Laine, Kristin Lauter, and Farinaz Koushanfar. XONN: XNOR-based oblivious deep neural network inference. In *USENIX Security*. USENIX Association, 2019.
- [93] Mike Rosulek. A brief history of practical garbled circuit optimizations. <https://simons.berkeley.edu/talks/mike-rosulek-2015-06-09>, <https://www.youtube.com/watch?v=FTxh908u9y8>, 2015. Accessed: September 7, 2024.
- [94] Bitu Darvish Rouhani, M. Sadegh Riazi, and Farinaz Koushanfar. DeepSecure: Scalable provably-secure deep learning. In *DAC*. ACM, 2018.
- [95] Henry N. Schuh, Arvind Krishnamurthy, David Culler, Henry M. Levy, Luigi Rizzo, Samira Khan, and Brent E. Stephens. Cc-nic: a cache-coherent interface to the nic. In *ASPLOS*, page 52–68. ACM, 2024.
- [96] Henry N. Schuh, Weihao Liang, Ming Liu, Jacob Nelson, and Arvind Krishnamurthy. Xenic: SmartNIC-accelerated distributed transactions. In *SOSP*. ACM, 2021.
- [97] Adi Shamir. How to share a secret. *CACM*, 22(11), 1979.
- [98] Sharemind. Sharemind MPC platform. <https://sharemind.cyber.ee/sharemind-mpc/multi-party-computation/>. Accessed: June 24, 2025.
- [99] Snowflake. Snowflake data clean rooms: Securely collaborate to unlock insights and value. <https://www.snowflake.com/en/blog/unlock-insights-with-snowflake-data-clean-rooms/>. Accessed: September 8, 2024.
- [100] Adithya Vadapalli, Ryan Henry, and Ian Goldberg. Duoram: A Bandwidth-Efficient distributed ORAM for 2- and 3-party computation. In *USENIX Security*, pages 3907–3924, Anaheim, CA, August 2023. USENIX Association.
- [101] Stephan van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. SGAXe: How SGX fails in practice. <https://sgaxeattack.com/>, 2020.
- [102] Stephan Van Schaik, Alex Seto, Thomas Yurek, Adam Batori, Bader AlBassam, Daniel Genkin, Andrew Miller, Eyal Ronen, Yuval Yarom, and Christina Garman. SoK: SGX.Fail: How stuff gets eXposed. In *S&P*, pages 4143–4162, 2024.
- [103] Tanya Verma and Sudheesh Singanamalla. Improving DNS privacy with Oblivious DoH in 1.1.1.1. <https://blog.cloudflare.com/oblivious-dns/>, 2020. Accessed: June 23, 2025.
- [104] Alexander Viand, Patrick Jattke, and Anwar Hithnawi. Sok: Fully homomorphic encryption compilers. In *S&P*, pages 1092–1108. IEEE, 2021.

- [105] Nikolaj Volgushev, Malte Schwarzkopf, Ben Getchell, Mayank Varia, Andrei Lapets, and Azer Bestavros. Conclave: secure multi-party computation on big data. In *EuroSys*. ACM, 2019.
- [106] Amanda Walker, Sarvar Patel, and Moti Yung. Helping organizations do more without collecting more data. Google Security Blog, 2019. <https://security.googleblog.com/2019/06/helping-organizations-do-more-without-collecting-more-data.html>.
- [107] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [108] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Global-scale secure multiparty computation. In *CCS*. ACM, 2017.
- [109] Jean-Luc Watson, Sameer Wagh, and Raluca Ada Popa. Piranha: A GPU platform for secure computation. In *USENIX Security*. USENIX, 2022.
- [110] Xingda Wei, Rong Chen, and Haibo Chen. Fast RDMA-based ordered Key-Value store using remote learned cache. In *OSDI*, pages 117–135. USENIX Association, November 2020.
- [111] Xingda Wei, Rongxin Cheng, Yuhan Yang, Rong Chen, and Haibo Chen. Characterizing off-path SmartNIC for accelerating distributed systems. In *OSDI*, pages 987–1004. USENIX Association, 2023.
- [112] Xingda Wei, Zhiyuan Dong, Rong Chen, and Haibo Chen. Deconstructing RDMA-enabled distributed transactions: Hybrid is better! In *OSDI*, pages 233–251, Carlsbad, CA, October 2018. USENIX Association.
- [113] Sophia Yakoubov. A gentle introduction to Yao’s garbled circuits, 2017. <http://web.mit.edu/sonka89/www/papers/2017ygc.pdf>.
- [114] Andrew C. Yao. Protocols for secure computations. In *FOCS*. IEEE, 1982.
- [115] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. In *EUROCRYPT*. Springer, 2015.
- [116] Wenting Zheng. *Sharing without Showing: Building Secure Collaborative Systems*. PhD thesis, University of California, Berkeley, 2020.
- [117] Wenting Zheng, Ryan Deng, Weikeng Chen, Raluca Ada Popa, Aurojit Panda, and Ion Stoica. Cerebro: A platform for Multi-Party cryptographic collaborative learning. In *USENIX Security*, pages 2723–2740. USENIX Association, 2021.